



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 1

“Puertos de Entrada/Salida”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de febrero de 2025

INTRODUCCIÓN

CodeVisionAVR

Es una herramienta clásica y muy potente en el mundo de los sistemas embebidos, específicamente diseñada para microcontroladores de la arquitectura AVR de Atmel. Es mucho más que un simple compilador; es un Entorno de Desarrollo Integrado (IDE) que facilita enormemente la configuración del hardware.

A diferencia de programar directamente en Atmel Studio o usando el framework de Arduino, CodeVision destaca por su Automatic Program Generator (CodeWizardAVR).

- **CodeWizardAVR:** Es una herramienta visual donde seleccionas qué periféricos quieres usar (Timers, UART, ADC, LCD, etc.) y el programa genera automáticamente todo el código de inicialización en C. Esto ahorra horas de lectura de hojas de datos (datasheets).
- **Compilador C ANSI:** Está altamente optimizado para la arquitectura AVR, lo que genera archivos “.hex” muy compactos.
- **Librerías integradas:** Incluye soporte nativo para pantallas LCD alfanuméricas y gráficas, sensores de temperatura (como el DS1820), buses I2C, SPI y más.

Cuando se trabaja con CodeVision, el proceso suele seguir este orden:

1. **Configuración del Chip:** Seleccionas el modelo (ej. ATmega328P) y la frecuencia del reloj.
2. **Uso del Wizard:** Configuras pines de entrada/salida y funciones especiales.
3. **Escritura de Lógica:** El Wizard te entrega un archivo .c con una estructura lista. Tú solo escribes tu lógica dentro del bucle while(1).
4. **Compilación y Programación:** El software genera el archivo binario y permite grabarlo directamente en el chip si tienes un programador compatible (como el AVRISP mkII o el USBasp).

Tabla comparativa.

Característica	CodeVisionAVR	Arduino IDE	Atmel/Microchip Studio
Dificultad	Media (Ideal para aprender hardware)	Baja (Abstracción total)	Alta (Profesional/Bare-metal)
Control	Total sobre los registros	Limitado por librerías	Total
Productividad	Muy alta por su Wizard	Alta por comunidad	Media (Configuración manual)

Es importante mencionar que es un software comercial (de pago), aunque tiene versiones:

- **Evaluation:** Gratuita, pero limita el tamaño del código generado (generalmente a 4KB).
- **Standard/Advanced:** Versiones completas para uso profesional sin restricciones.

Microcontrolador ATMEGA8535

Es uno de los microcontroladores más icónicos de la familia AVR de 8 bits. Es especialmente apreciado en entornos educativos y de prototipado porque es el sucesor directo del clásico AT90S8535, manteniendo una distribución de pines muy cómoda para trabajar en placas de pruebas.

Características:

- **Memoria:**
 - Flash: 8 KB (donde se guarda el código).
 - SRAM: 512 Bytes (para variables temporales).
 - EEPROM: 512 Bytes (memoria no volátil para datos que deben persistir tras apagarlo).
- **Velocidad:** Hasta 16 MHz (proporcionando cerca de 16 MIPS).
- **Voltaje de operación:** 4.5V a 5.5V (versión estándar) o 2.7V a 5.5V (versión L).
- **Pines de E/S:** 32 pines programables divididos en 4 puertos (A, B, C y D).

Periféricos integrados:

Lo que hace al ATmega8535 muy versátil para proyectos de control es su conjunto de herramientas internas:

- **Convertor Analógico-Digital (ADC):** Tiene 8 canales con resolución de 10 bits. Ideal para leer sensores analógicos.
- **Timers/Contadores:** * Dos de 8 bits.
 - Uno de 16 bits (útil para tiempos muy precisos o PWM de alta resolución).
- **Canales PWM:** 4 canales para control de motores o intensidad de luces.
- **Comunicación:**
 - UART: Para comunicación serial (RS232).
 - SPI e I2C (TWI): Para conectar pantallas, memorias externas u otros sensores digitales.
- **Comparador Analógico:** Compara dos voltajes de entrada directamente en el hardware.

El ATmega8535 utiliza la arquitectura Harvard (memorias de datos y programa separadas) y un conjunto de instrucciones RISC (Reduced Instruction Set Computer).

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 1, el circuito a armar es el siguiente:

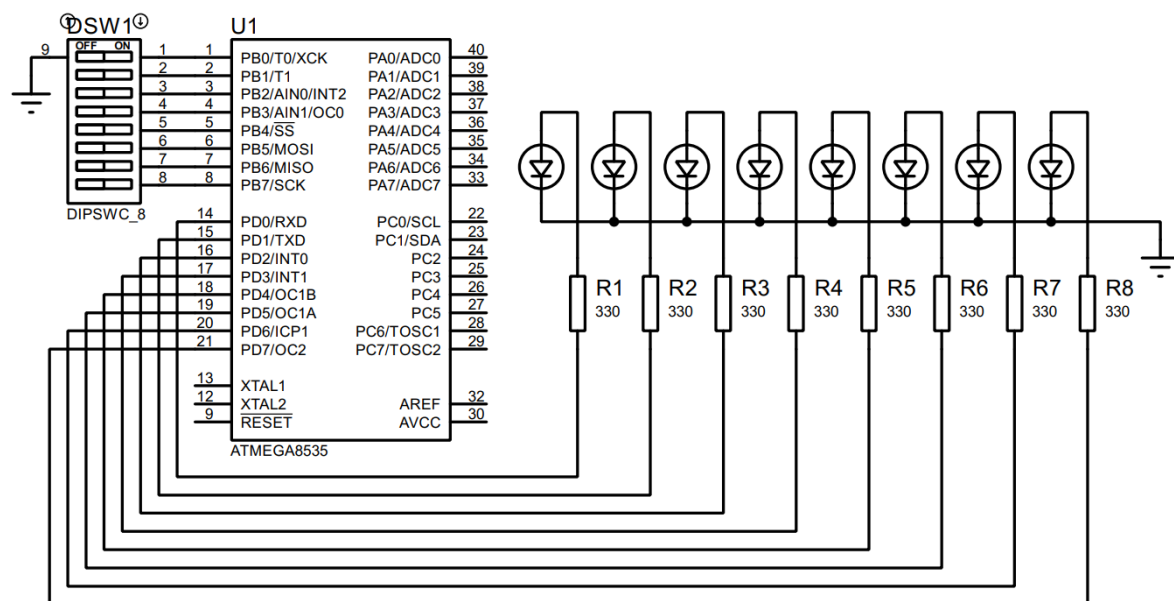


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, todo el registro B se configuran como de entrada, activando las resistencias de pull-up, mientras que el registro D como de salida, con un valor inicial de cero.

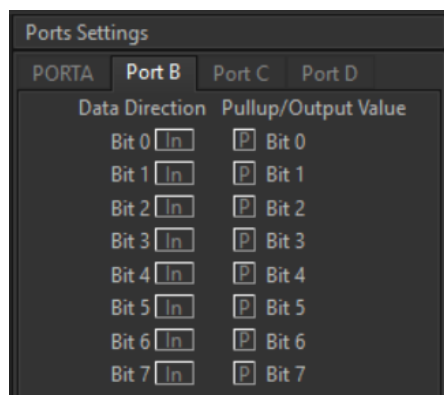


Imagen 2. Configuración del registro B.

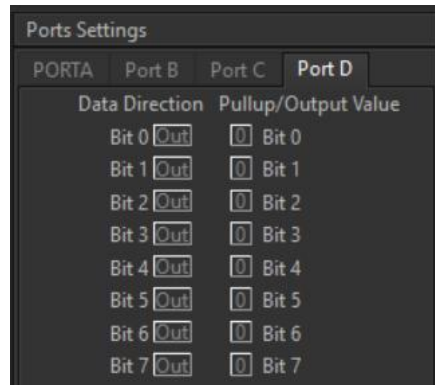


Imagen 3. Configuración del registro D.

Código del circuito de CodeVision:

```

1  /******
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Práctica 1 'Puertos de Entrada/Salida'
8  Version : 1.0
9  Date    : 07/02/2026
10 Author  : García Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>

```

```

26 // Declare your global variables here
27
28 void main(void) {
29     // Declare your local variables here
30
31     // Input/Output Ports initialization
32     // Port A initialization
33     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
34     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
35     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
36     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
37
38     // Port B initialization
39     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
40     DDRB=(0<<ddb7) | (0<<ddb6) | (0<<ddb5) | (0<<ddb4) | (0<<ddb3) | (0<<ddb2) | (0<<ddb1) | (0<<ddb0);
41     // State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
42     PORTB=(1<<PORTB7) | (1<<PORTB6) | (1<<PORTB5) | (1<<PORTB4) | (1<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
43
44     // Port C initialization
45     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
46     DDRC=(0<<ddc7) | (0<<ddc6) | (0<<ddc5) | (0<<ddc4) | (0<<ddc3) | (0<<ddc2) | (0<<ddc1) | (0<<ddc0);
47     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
48     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

```

```

50 // Port D initialization
51 // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
52 DDRD=(1<<DDD7) | (1<<DDD6) | (1<<DDD5) | (1<<DDD4) | (1<<DDD3) | (1<<DDD2) | (1<<DDD1) | (1<<DDD0);
53 // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
54 PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
55
56 // Timer/Counter 0 initialization
57 // Clock source: System Clock
58 // Clock value: Timer 0 Stopped
59 // Mode: Normal top=0xFF
60 // OC0 output: Disconnected
61 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
62 TCNT0=0x00;
63 OCR0=0x00;

```

```

65 // Timer/Counter 1 initialization
66 // Clock source: System Clock
67 // Clock value: Timer1 Stopped
68 // Mode: Normal top=0xFFFF
69 // OC1A output: Disconnected
70 // OC1B output: Disconnected
71 // Noise Canceler: Off
72 // Input Capture on Falling Edge
73 // Timer1 Overflow Interrupt: Off
74 // Input Capture Interrupt: Off
75 // Compare A Match Interrupt: Off
76 // Compare B Match Interrupt: Off
77 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
78 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
79 TCNT1H=0x00;
80 TCNT1L=0x00;
81 ICR1H=0x00;
82 ICR1L=0x00;
83 OCR1AH=0x00;
84 OCR1AL=0x00;
85 OCR1BH=0x00;
86 OCR1BL=0x00;

```

```

88 // Timer/Counter 2 initialization
89 // Clock source: System Clock
90 // Clock value: Timer2 Stopped
91 // Mode: Normal top=0xFF
92 // OC2 output: Disconnected
93 ASSR=0<<AS2;
94 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
95 TCNT2=0x00;
96 OCR2=0x00;
97
98 // Timer(s)/Counter(s) Interrupt(s) initialization
99 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
100
101 // External Interrupt(s) initialization
102 // INT0: Off
103 // INT1: Off
104 // INT2: Off
105 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
106 MCUCSR=(0<<ISC2);
107
108 // USART initialization
109 // USART disabled
110 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

112 // Analog Comparator initialization
113 // Analog Comparator: Off
114 // The Analog Comparator's positive input is
115 // connected to the AIN0 pin
116 // The Analog Comparator's negative input is
117 // connected to the AIN1 pin
118 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
119 SFIOR=(0<<ACME);
120
121 // ADC initialization
122 // ADC disabled
123 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
124
125 // SPI initialization
126 // SPI disabled
127 SPCR=(0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
128
129 // TWI initialization
130 // TWI disabled
131 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

```

133 while (1) {
134     // Place your code here
135     PORTD = PINB; // lo que entra es igual a lo que sale
136 }
137 }

```

Circuito físico

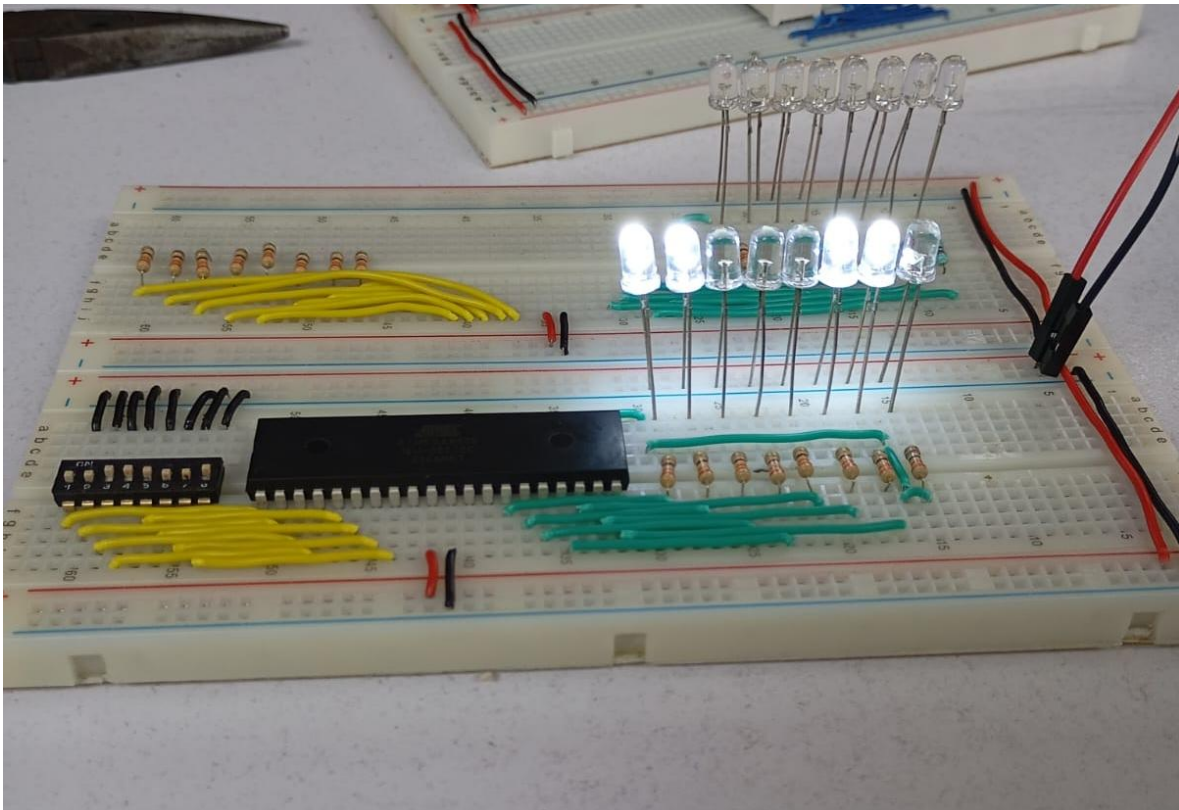


Imagen 3. Circuito físico de la práctica 1.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

A pesar de que la práctica es muy sencilla, resulta ser de mucha ayuda porque aprendemos a cómo usar los puertos de la ATMEGA8535, usarlos tanto como de salida como de entrada. Por otra parte, nos introduce a la herramienta CodeVision para poder programar el microcontrolador. Esta resulta ser muy sencilla de usar, pues nos permite configurar cada uno de los registros del microcontrolador de forma muy intuitiva. Entre las configuraciones, esta la de indicar que bits del registro serán de salida o entrada, donde para el primero se podrá indicar el valor inicial de bit, mientras que para el segundo se podrá activar o desactivar la resistencia de pull-up.

Por último, CodeVision usa lenguaje C para poder escribir el programa que se grabará en el microcontrolador, haciendo este proceso algo muy sencillo.

- **Meléndez Macedonio Rodrigo**

Esta primer practica fue fácil de desarrollar, sin embargo, lo verdaderamente esencial fue la introducción del software de CodeVisionAVR, pues con ayuda de esta práctica aprendimos lo básico que se necesita al momento de utilizar el microcontrolador ATMEGA8535 en conjunto con el software antes mencionado, de modo que todo el proceso para configurar los puertos de entrada y salida resulto bastante sencillo e intuitivo, así como programar el comportamiento de nuestro circuito con ayuda del lenguaje C, ya que solo basto con unas cuantas líneas de código para lograr que funcionará el circuito que hicimos. Finalmente, al momento de programar el ATMEGA8535 no tuvimos ningún problema con los drivers y todo salió de acuerdo con lo esperado.

BIBLIOGRAFÍA

- (2023). *CodeVisionAVR User Manual*. Recuperado el 09 de febrero de 2026. <http://www.hpinfotech.ro/cvavr-documentation.html>
- (s.f.). *Microchip Technology. ATmega8535 Datasheet (Complete)*. Recuperado el 09 de febrero de 2026. <https://www.alldatasheet.com.mx/datasheet-pdf/pdf/164169/ATMEL/ATMEGA8535.html>